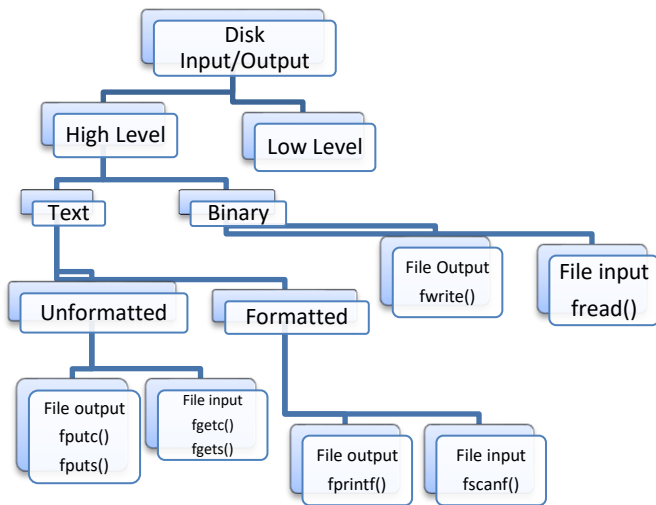


FILE HANDLING (CHAPTER -13)

1) What is a File ?

- C allows data to be stored permanently in a data file with the help of an extensive set of library functions for handling these data files. A file in C treated as a serial sequence or stream of bytes.

2) What are the types of Files ?



3) How to open & close a File ?

- To store data in the file, the data needs to be transferred from the computer's memory to the data file in the hard disk. Output data does not go directly to the hard disk; instead, data goes to a temporary storage area in memory called a buffer. When the buffer becomes full, the data is output to the storage disk for permanent storage. Similarly, data inputs from a disk go to an input buffer 1st then loaded into the main memory. The use of a memory buffer makes data transfer more efficient.

4) How many buffer memories are there ?

- 2 types of buffer memories are there: a) Input Buffer Memory b) Output Buffer Memory.

5) What is File output & File input in C language ?

- When data is transferred from a program to a file, it is called a file output & when the data is transferred from a file to a program, it is called file input.

6) How to declare a file pointer in C ?

- To declare a file pointer **FILE** keyword is used (in capital). This **FILE** is a pointer type of variable to point to a particular variable. This file pointer is also known as stream pointer.

Syntax for creating a file pointer –

FILE name of the variable.

eg. – FILE *fp; --> Here fp is a pointer variable to store the address of the structure type FILE. fp also acts as a link between the program & the OS.

7) How to open a file ?

- After creating file pointer, it is used to open the file in diff. mode i.e. read mode, write mode etc.

The function **fopen()** is used to open a file.

The syntax of opening a file –

fp = fopen("filename", "mode");

eg. – fp = fopen(abc.txt, "r"); → here abc.txt file will open in buffer memory in "r" for read mode.

8) Describe mode of opening a text file ?

File Type	Description
"r"	Opening an existing file for reading only.
"w"	Opening a "new file" for writing only. Or overwrite any existing file with the same name.
"a"	Opening an existing file for "appending" . A new file gets created in case the file does not exist.
"r+"	Open an existing file for reading, writing & modification.
"w+"	Open an new file for reading, writing & modification. This will overwrite any existing file with the same name.
"a+"	Open an existing file for reading & appending (but modification not allowed). A new file gets created in case the file-name does not exist.

9) How to close a file in C ?

- **fclose()** function is used to close the file from the buffer memory as well as main memory.

The syntax for closing file is –

`fclose(name of the variable);`

eg. – `fclose(fp);` → here `fp` is the pointer variable to point the file.

10) How to print string or get output string from a text file in C using `fputs()`?

- To print the whole lines or whole paragraph from a text file **`fputs()`** function is used. It is similar to **`puts()`** function which is used to print string on the output string. **`fputs()`** needs 2 arguments as a parameter.

The Syntax of **`fputs()`** function is –

`fputs(string,name of the variable);`

eg. –

`FILE *fp;`

`char s[80];`

`gets(s);`

`fputs(s,fp);` → `s` is a character array type variable to store the string inputted by the user within the text file & `fp` is a pointer type of variable to point the name of the file.

Note - the `fputs()` function continues to write the string to the file it gets a '\0' character, which it does not write to the file. Though the `fputs()` function writes an entire string to a file but it does not insert a newline character at the end of the string.

11) How to print string or get output string from a text file in C using `fprintf()`?

- **`fprintf()`** is used to print formatted data. **`fprintf()`** function has same syntax as the `printf()` has. However, `FILE` pointer also needs to be included to indicate an output to a file.

The syntax of `fprintf()` is –

`fprintf(name of the variable,control string,data list);`

eg.- `fprintf(fp,"print whatever you want",variable to print data);`

12) How to print string or get output string from a text file in C using `fputc()`?

- To print a single character from a text file **`fputc()`** function is used. It is similar to **`putc()`** function which is used to print string on the output string. **`fputc()`** needs 2 arguments as a parameter.

The Syntax of **`fputc()`** function is –

`fputc(character,name of the variable);`

eg. –

`FILE *fp;`

`char s;`

`getchar(s);`

`fputc(s,fp);` → `s` is a character variable to store a character inputted by the user within the text file & `fp` is a pointer type of variable to point the name of the file.

Note - the `fputc()` function continues to write a single character to the file it gets a '\0' character, which it does not write to the file. Though the `fputc()` function writes a single character to a file but it does not insert a newline character at the end of the string.

13) How to get input single character at a time from a text file in C using `fgetc()`?

- To get a single character from a text file **`fgetc()`** function is used. It is similar to **`getc()`** function which is used to get a single character from a text file on the output string. **`fgetc()`** needs 1 arguments as a parameter.

The Syntax of **`fgetc()`** function is –

`fgetc(name of the variable);`

eg. –

`FILE *fp;`

`char ch;`

`ch=fgetc(fp);` → `ch` is a character variable to get a single character getting from the text file & `fp` is a pointer type of variable to point the name of the file.

14) How to detect end of the file in C ?

- While reading from a data file, the data input operation should stop when the end of the file is reached. This is similar to reading a string which stops when the special '\0' character is reached. **EOF** is a keyword in C file which is used to check whether file has reached last character i.e. end of the file or not.

15) How to get string at a time from a text file in C using `fgets()`?

- To get a input string from a text file **`fgets()`** function is used. It is similar to **`gets()`** function which is used to get a input string from a text file on the output string. **`fgets()`** needs 3 arguments as a parameter.

The Syntax of **`fgets()`** function is –

`fgets(string,maximum length,name of the variable);`

eg. –

`FILE *fp;`

`char ch[len];`

`fgets(ch,len,fp);` → `ch` is a character array to get a string getting from the text file & `fp` is a pointer type of variable to point the name of the file.

16) What is the function `fscanf()` in C ?

- Formatted input is necessary as the `fgetc()` & `fgets()` functions can only input character or string data from a file. To input formatted text & number values we have to use `fscanf()`.

`fscanf()` is similar syntax like `scanf()`, but `FILE` pointer variable has to be included to indicate an input from a file. The function usually returns an integer value specifying the no. of values successfully read, otherwise it returns an EOF value in case no input is read or an error occurs during an input.

The syntax of `fscanf()` is –

`fscanf(file pointer, control string , variable list)`

eg.- `fscanf(fp,"%d %d",a,b);` → here `fp` is a file pointer, `%d %d` → is control string, `a,b` → variable listing.

17) What are the disadvantages of Text File ?

- A character stored as a character or a string as a series of characters with each character occupying a single byte, however integers, floats & other numbers are stored as numerical values, but as a string or character. Thus the suppose no. 1456 will not be stored as an int type data occupying 2 bytes but as a set of 4 characters as 1456, with each digit occupying a single byte i.e. in all 4 bytes are required to store the no. 1456 in text mode.
- When data is entered in a binary mode, binary data as it appears in memory is transferred directly to the file.
- Characters lie '\n' or '\0' which have a significance in a text file, does not have any special significance in a binary file. No data transformation is involved. Thus an int type data will requires 2 bytes & float type data will require 4 bytes. This characteristic of binary files makes them more compact & efficient to use.
- Data in a binary file is stored in an unformatted manner, hence cannot be displayed directly using text editor like notepad & wordpad.

18) Describe mode of opening a binary file ?

File Type	Description
"rb"	Opening an existing binary file for reading only.
"wb"	Opening a "new binary file" for writing only. Or overwrite any existing file with the same name.
"ab"	Opening an existing binary file for "appending" . A new file gets created in case the file does not exist.
"rb+", r+b	Open an existing binary file for reading, writing & modification.
"wb+", w+b	Open an new binary file for reading, writing & modification. This will overwrite any existing file with the same name.
"ab+", a+b	Open an existing binary file for reading & appending(but modification not allowed). A new file gets created in case the file-name does not exist.

19) How get data as output on screen using fwrite() using Binary file ?

- The syntax of fwrite() is –
 fwrite(&variable, variable size, number of items, file pointer);
 &variable → indicates the address of the variable to output.
 variable size → indicates no. of bytes the variable occupies. This can be automatically calculated using sizeof() function.
 no. of items → This is an integer no. indicating the no. of variables one wants to write at a time.
 file pointer → indicates FILE pointer to the file.
 fwrite() operates by writing a specified no. of objects to a file, where each object is given no. of bytes long. The function returns the count of the no. of items actually written to the file. In case of an unsuccessful write operation, the return value will be less than the no. of item values.

20) How get data input using fread() from Binary file ?

- The syntax of fread() is –
 fread(&variable, variable size, number of items, file pointer);
 &variable → indicates the address of the variable to output.
 variable size → indicates no. of bytes the variable occupies. This can be automatically calculated using sizeof() function.
 no. of items → This is an integer no. indicating the no. of variables one wants to write at a time.
 file pointer → indicates FILE pointer to the file.
 fread() operates by reading a specified no. of objects from a file, where each object is given no. of bytes long. The function returns an integer indicate that the count of the no. of objects successfully read from the file. In case there is insufficient data in the file or if an error occurs, the return value will be less than the no. of objects.

21) Few important things for end of a file ?

- fgetc() function returns EOF when the end of a file is reached.
- fgets() function returns NULL when the end of a file is reached.
- fprintf() function returns EOF when the end of a file is reached.

22) Diff. between Text File & Binary File ?

TEXT FILE	BINARY FILE
1. NULL & newline have special significance.	1. NULL & newline have no special significance.
2. Data can be formatted while storing the data in a text file.	2. No formatting is applied on the data when they are stored in a binary file.
3. to read "r" mode is used. to write "w" mode is used. to append "a" mode is used.	3. to read "rb" mode is used. to write "wb" mode is used. to append "ab" mode is used.
4. Output & input functions for text files include fprintf(), fscanf() etc.	4. Output & input functions for text files include fwrite(), fread() etc.
5. As there is a data transformation during a read/write operation, these files are slower to run than binary files.	5. As no data transformation during a read/write operation, these files are faster to run than text files.

23) What is the utility of rewind() function ?

- To go back to the beginning of a file without closing the file, by using the rewind() function.

24) Which functions are used to move around in a file & access selected data randomly ?

- fseek() & ftell() functions are used to move around in a file & access selected data randomly.

fseek() → is used to move the file position to a desired location within the file.

The syntax is –

fseek(file pointer, offset, position);

offset is a numeric value of type long & indicates the no. of bytes to move from the location indicated by position. Offset can be positive or negative value. A positive value indicates moving forward in the file & negative value indicates moving backward in the file.

position is an integer no. & can take any values like :-

value =0 beginning of file.

value =1 current position of file.

value =2 End of file.

The fseek() function executes successfully then it returns a zero. Else if it moves past the beginning or the end of the file, then it returns an EOF value i.e. -1.

ftell() → is used to get the current file position in the file. The function needs the filepointer of the file to use & returns a value of type long, that corresponds to the current file position.

The syntax is –

variable= ftell(file pointer);

eg- int x;

x=ftell(fp);

File operation	Declaration & Description
fopen() – To open a file	Declaration: FILE *fopen (const char *filename, const char *mode) fopen() function is used to open a file to perform operations such as reading, writing etc. In a C program, we declare a file pointer and use fopen() as below. fopen() function creates a new file if the mentioned file name does not exist. FILE *fp; fp=fopen ("filename", "mode"); Where, fp – file pointer to the data type "FILE". filename – the actual file name with full path of the file. mode – refers to the operation that will be performed on the file. Example: r, w, a, r+, w+ and a+. Please refer below the description for these mode of operations.
fclose() – To close a file	Declaration: int fclose(FILE *fp); fclose() function closes the file that is being pointed by file pointer fp. In a C program, we close a file as below. fclose (fp);
fgets() – To read a file	Declaration: char *fgets(char *string, int n, FILE *fp) fgets function is used to read a file line by line. In a C program, we use fgets function as below. fgets (buffer, size, fp); where, buffer – buffer to put the data in. size – size of the buffer fp – file pointer
fprintf() – To write into a file	Declaration: int fprintf(FILE *fp, const char *format, ...); fprintf() function writes string into a file pointed by fp. In a C program, we write string into a file as below. fprintf (fp, "some data"); or fprintf (fp, "text %d", variable_name);

INBUILT FUNCTIONS FOR FILE HANDLING IN C LANGUAGE:

C programming language offers many inbuilt functions for handling files. They are given below. Please click on each function name below to know more details, example programs, output for the respective file handling function.

File handling functions	Description
fopen ()	fopen () function creates a new file or opens an existing file.
fclose ()	fclose () function closes an opened file.

getw ()	getw () function reads an integer from file.
putw ()	putw () functions writes an integer to file.
fgetc ()	fgetc () function reads a character from file.
fputc ()	fputc () functions write a character to file.
gets ()	gets () function reads line from keyboard.
puts ()	puts () function writes line to o/p screen.
fgets ()	fgets () function reads string from a file, one line at a time.
fputs ()	fputs () function writes string to a file.
feof ()	feof () function finds end of file.
fgetchar ()	fgetchar () function reads a character from keyboard.
fprintf ()	fprintf () function writes formatted data to a file.
fscanf ()	fscanf () function reads formatted data from a file.
fputchar ()	fputchar () function writes a character onto the output screen from keyboard input.
fseek ()	fseek () function moves file pointer position to given location.
ftell ()	ftell () function gives current position of file pointer.
rewind ()	rewind () function moves file pointer position to the beginning of the file.
getc ()	getc () function reads character from file.
getch ()	getch () function reads character from keyboard.
getche ()	getche () function reads character from keyboard and echoes to o/p screen.
getchar ()	getchar () function reads character from keyboard.
putc ()	putc () function writes a character to file.
putchar ()	putchar () function writes a character to screen.
printf ()	printf () function writes formatted data to screen.
sprintf ()	sprintf () function writes formatted output to string.
scanf ()	scanf () function reads formatted data from keyboard.
sscanf ()	sscanf () function Reads formatted input from a string.
remove ()	remove () function deletes a file.
fflush ()	fflush () function flushes a file.